Enhancing Agentic Coding Workflows with Structured Representations

2026-04-04

What’s the value of agentic coding?

What’s the value of agentic coding?

- Increase the speed with which a company’s tech platform is improved

What’s the value of agentic coding?

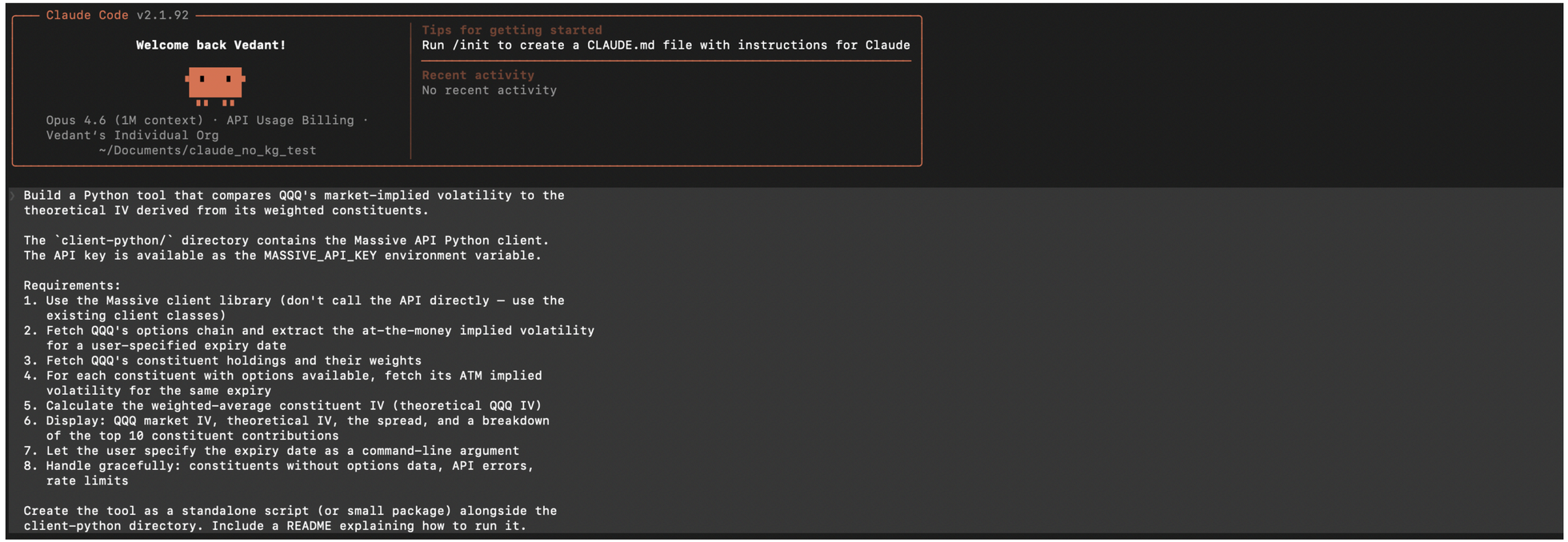
- Increase the speed with which a company’s tech platform is improved
- Decrease the costs of maintaining a technology platform

What’s the value of agentic coding?

- Increase the speed with which a company’s tech platform is improved
- Decrease the costs of maintaining a technology platform
- Improve the function of the technology platform to meet changing business needs

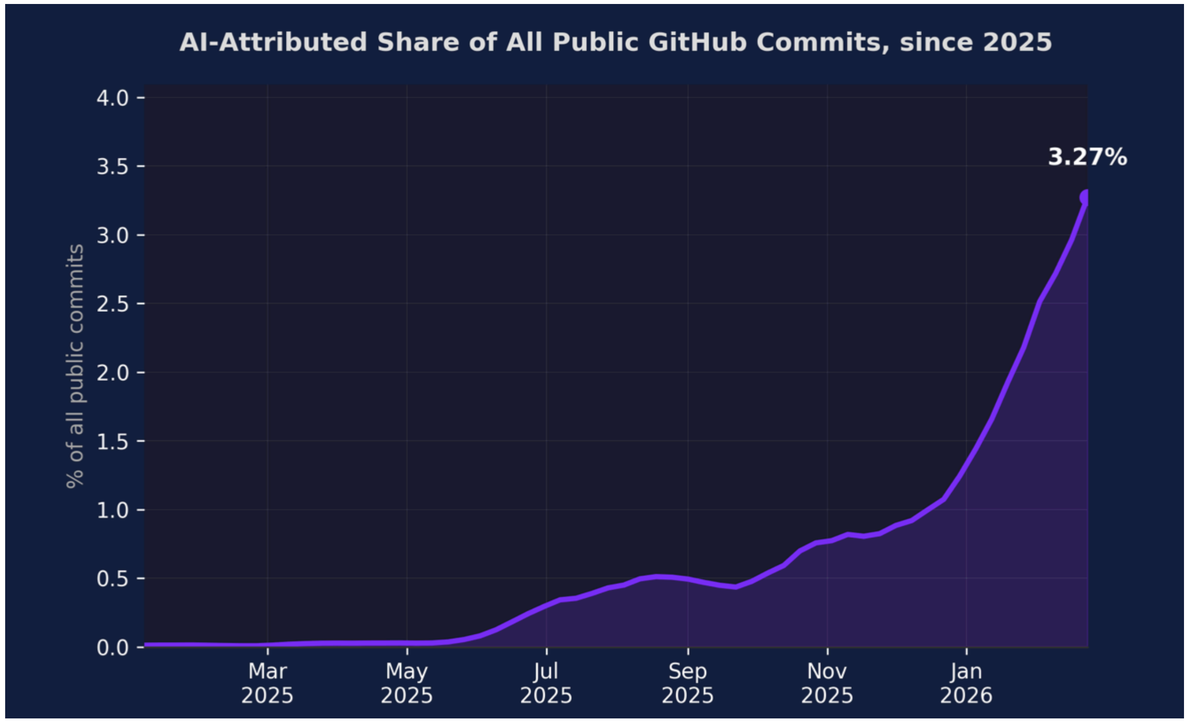
AI Coding platforms promise to do this for you

- Change the way we code from being hands on keyboard, in a coding language to being driven by natural language prompts

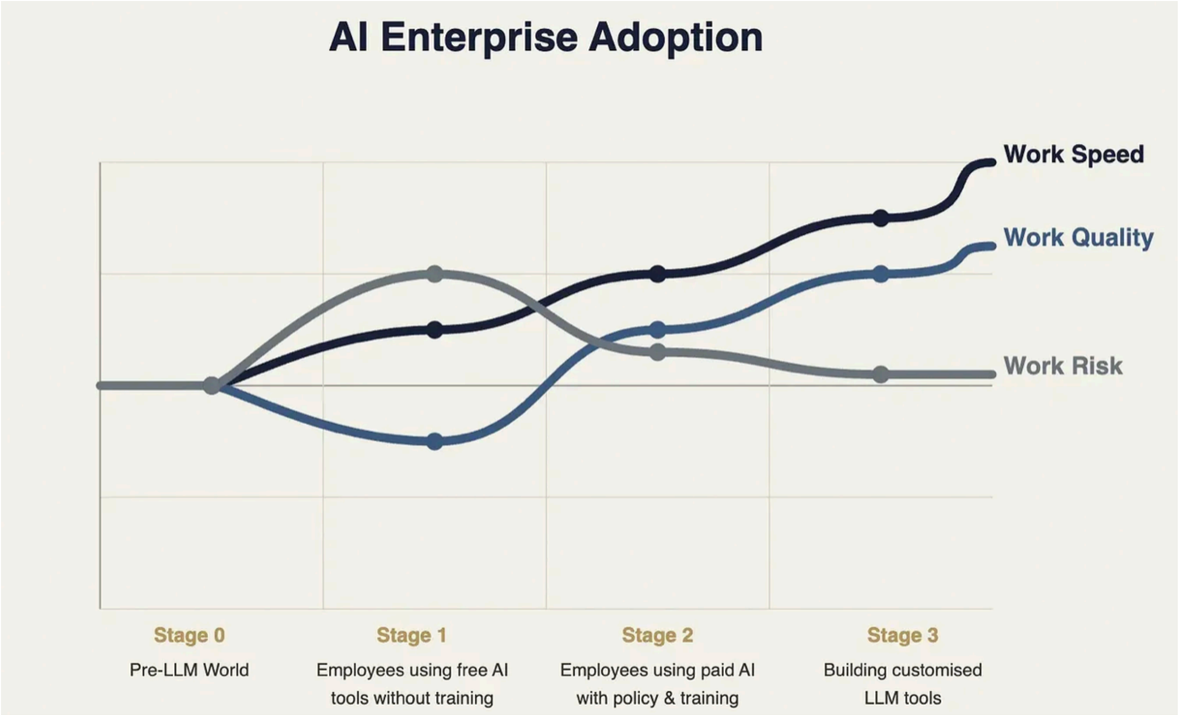


How is this being realized today?

- More git commits being written by AI



- More git commits being written by AI



Plots from Charlie Taggart and Towards AI Medium pages

What are the challenges?

- Feedback I've had for AI tools or received from peers using AI coding tools (paraphrased):
  - "That's a big MR."
  - "Claude created a bunch of code but then deleted my tests to claim it achieved its goals"
  - "I have to direct [the LLM] where to look and where to edit or it will make unexpected change"

What’s the underlying problem?

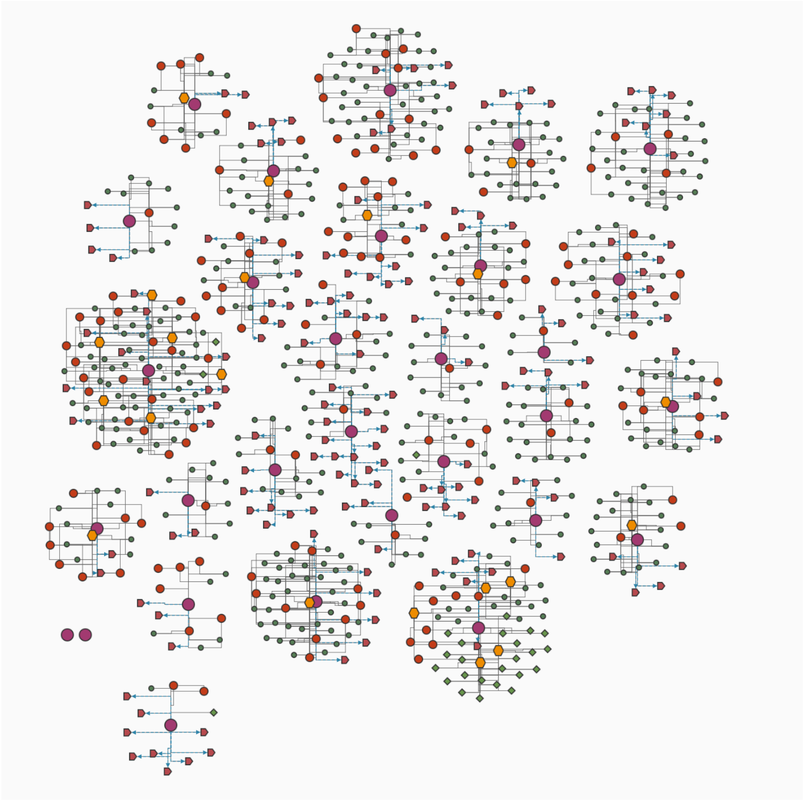
- Feedback I’ve had for AI tools or received from peers using AI coding tools (paraphrased):
  - “That’s a big MR.”
  - “Claude created a bunch of code but then deleted my tests to claim it achieved its goals”
  - “I have to direct [the LLM] where to look and where to edit or it will make unexpected change”
- Codebases are complicated, and often, multiple ways of updating a codebase can achieve the same end - but not all approaches to coding are equivalent

What’s the underlying problem?

- Codebases are complicated, and often, multiple ways of updating a codebase can achieve the same end - but not all approaches to coding are equivalent
  - Example: Create a dark mode for a website
  - You could manually update every single page to have a dark mode setting…
  - Or you could update a central CSS controller

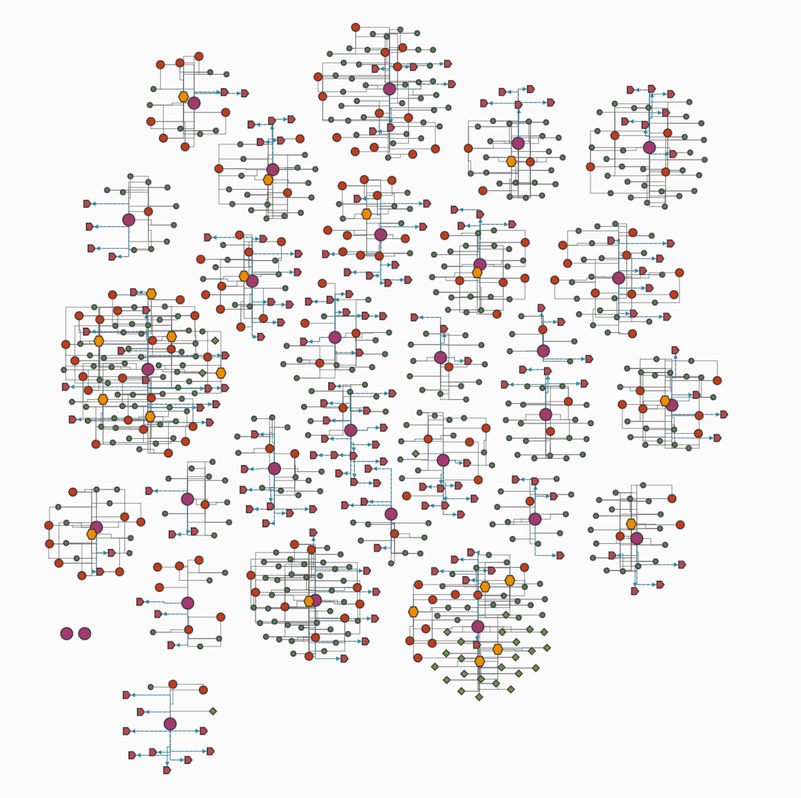
How can we guide AI to solving problems in the way we want them to be solved?

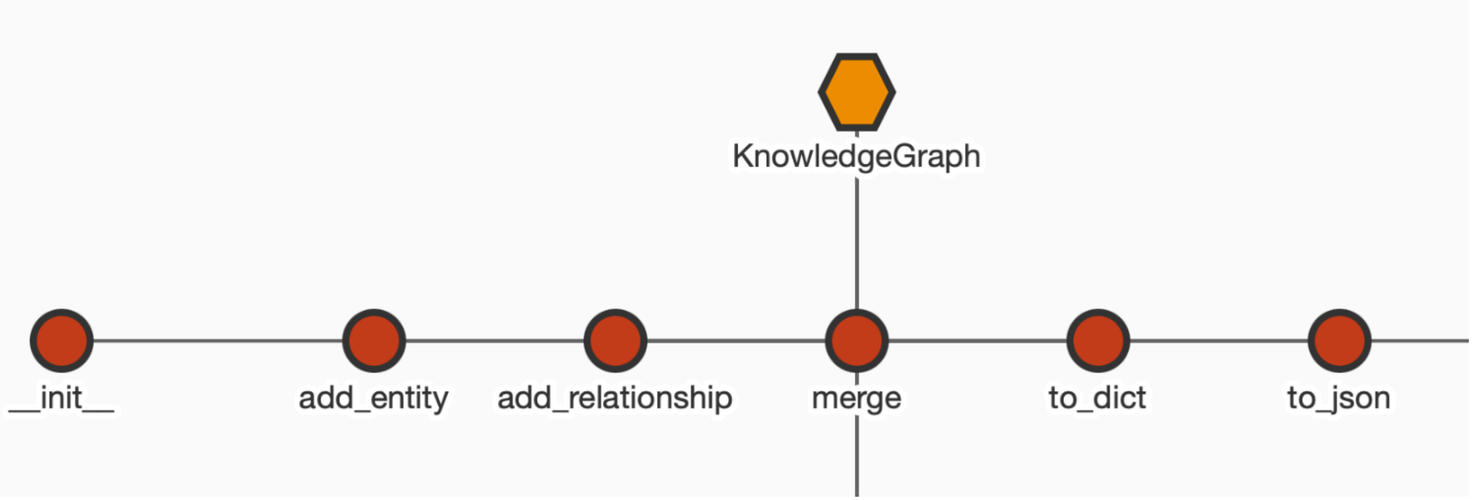
- Currently, LLMs understand codebases by reading every line of code, summarizing, and then using RegEx commands to try to figure out where to make changes
- However, here's an alternative look



Let’s zoom in a little

We can now see the entities and their relationship





This is probably closer to how I understand a codebase

Why is this important?

I can now explicitly indicate where a model should look and how it should edit

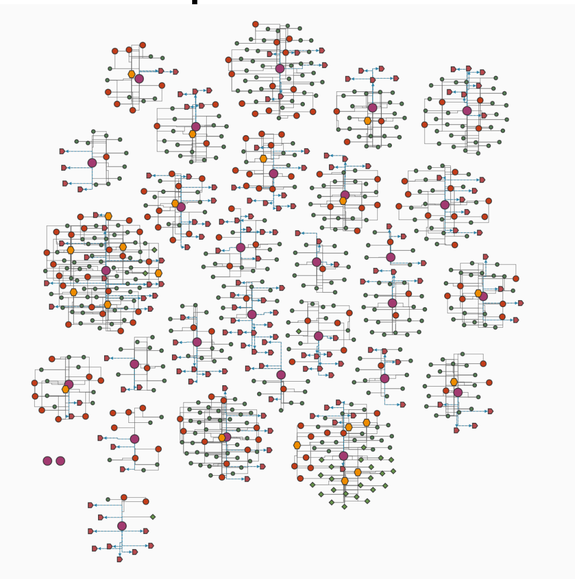
- From PM - JIRA-4901: Update codebase to figure out the distance between entities. Requirements include taking in a knowledge graph and two entities in the graph and finding the distance between the two

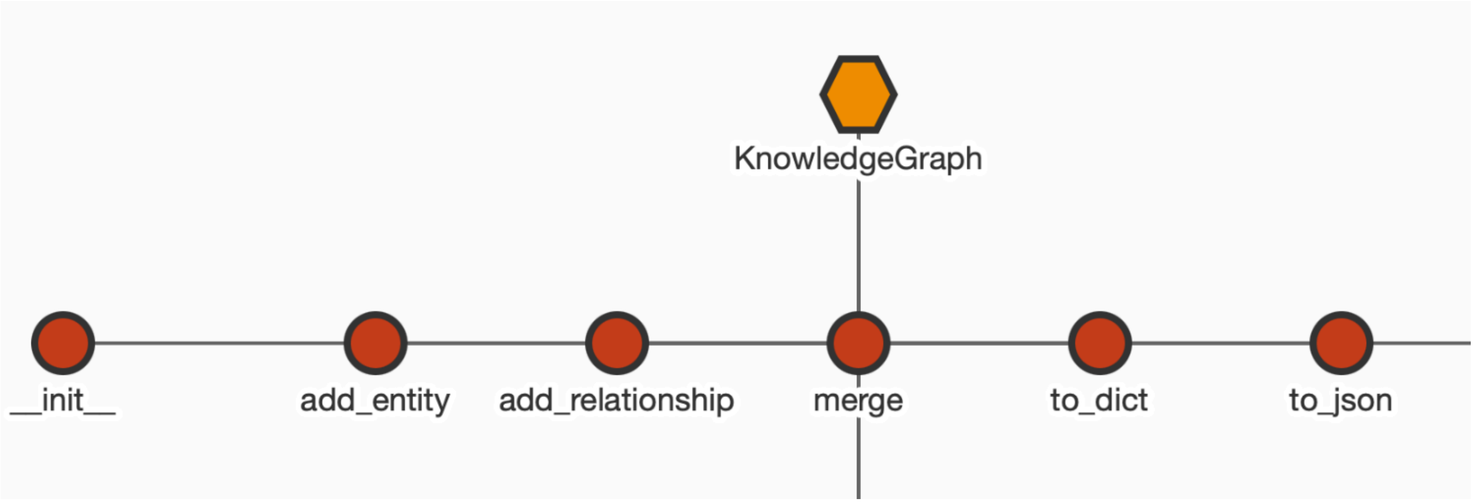
Why is this important?

I can now explicitly indicate where a model should look and how it should edit

- From PM - JIRA-4901: Update codebase to figure out the distance between entities. Requirements include taking in a knowledge graph and two entities in the graph and finding the distance between the two

- Execution planning - basic approach is to achieve functionality while minimizing impact



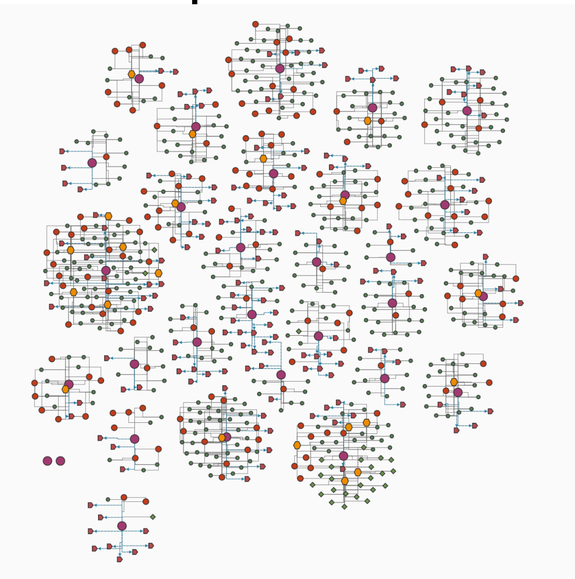


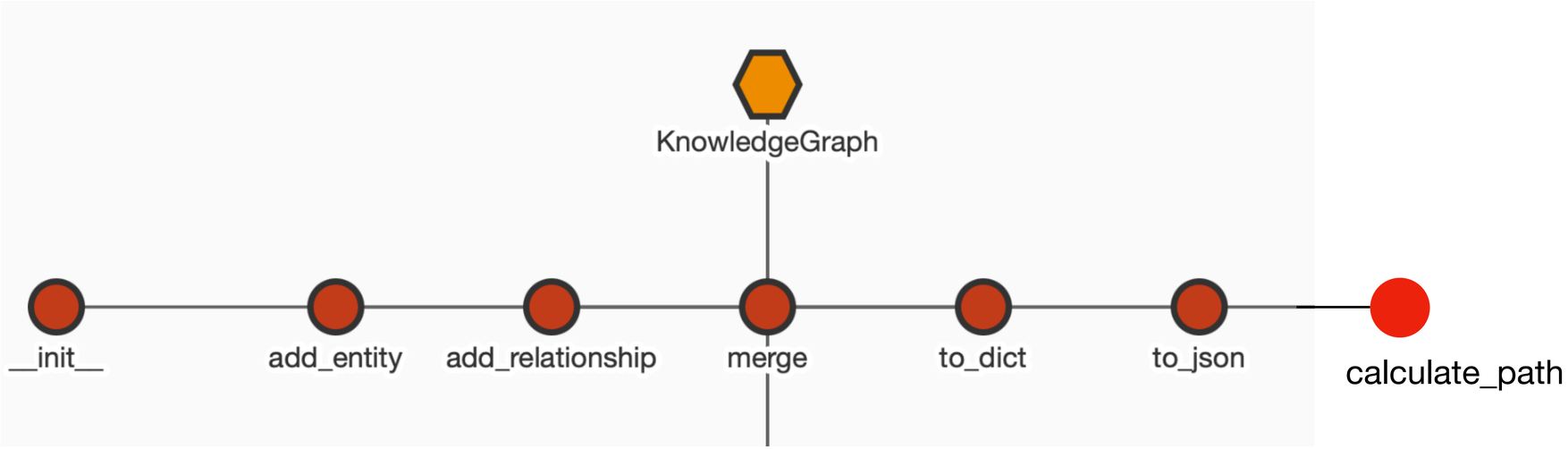
Why is this important?

I can now explicitly indicate where a model should look and how it should edit

- From PM - JIRA-4901: Update codebase to figure out the distance between entities. Requirements include taking in a knowledge graph and two entities in the graph and finding the distance between the two

- Execution planning - basic approach is to achieve functionality while minimizing impact

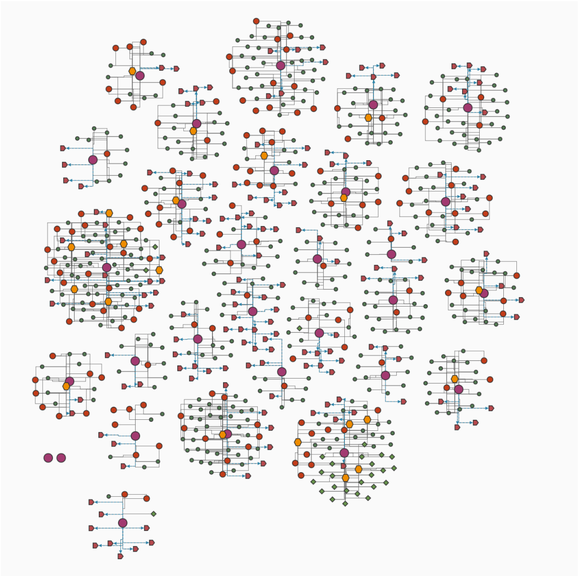


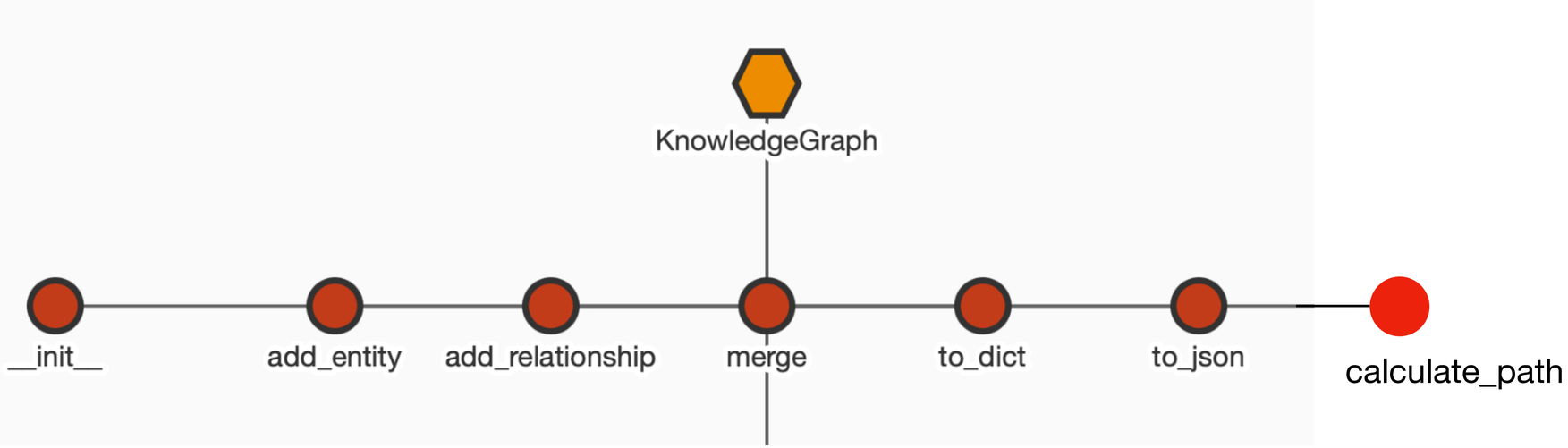


Why is this important?

I can now explicitly indicate where a model should look and how it should edit

- Execution planning - basic approach is to achieve functionality while minimizing impact





- Get stakeholder alignment on execution plan

Why is this important?

I can now explicitly indicate where a model should look and how it should edit

- Now I code.

Example use

- Task:

Build a Python tool that compares QQQ's market-implied volatility to the theoretical IV derived from its weighted constituents.
The `client-python/` directory contains the Massive API Python client.
The API key is available as the MASSIVE_API_KEY environment variable.
Requirements:

1. Use the Massive client library (don't call the API directly — use the existing client classes)
2. Fetch QQQ's options chain and extract the at-the-money implied volatility for a user-specified expiry date
3. Fetch QQQ's constituent holdings and their weights
4. For each constituent with options available, fetch its ATM implied volatility for the same expiry
5. Calculate the weighted-average constituent IV (theoretical QQQ IV)
6. Display: QQQ market IV, theoretical IV, the spread, and a breakdown of the top 10 constituent contributions
7. Let the user specify the expiry date as a command-line argument
8. Handle gracefully: constituents without options data, API errors, rate limits

Create the tool as a standalone script (or small package) alongside the client-python directory. Include a README explaining how to run it.

Example use

- Task:

Build a Python tool that compares QQQ's market-implied volatility to the theoretical IV derived from its weighted constituents.
The `client-python/` directory contains the Massive API Python client.
The API key is available as the MASSIVE_API_KEY environment variable.
Requirements:

1. Use the Massive client library (don't call the API directly — use the existing client classes)
2. Fetch QQQ's options chain and extract the at-the-money implied volatility for a user-specified expiry date
3. Fetch QQQ's constituent holdings and their weights
4. For each constituent with options available, fetch its ATM implied volatility for the same expiry
5. Calculate the weighted-average constituent IV (theoretical QQQ IV)
6. Display: QQQ market IV, theoretical IV, the spread, and a breakdown of the top 10 constituent contributions
7. Let the user specify the expiry date as a command-line argument
8. Handle gracefully: constituents without options data, API errors, rate limits

Create the tool as a standalone script (or small package) alongside the client-python directory. Include a README explaining how to run it.

How might we do this?

- Get the constituents of an ETF
- Get the options chain, the stock prices, and do the corresponding math

Example use

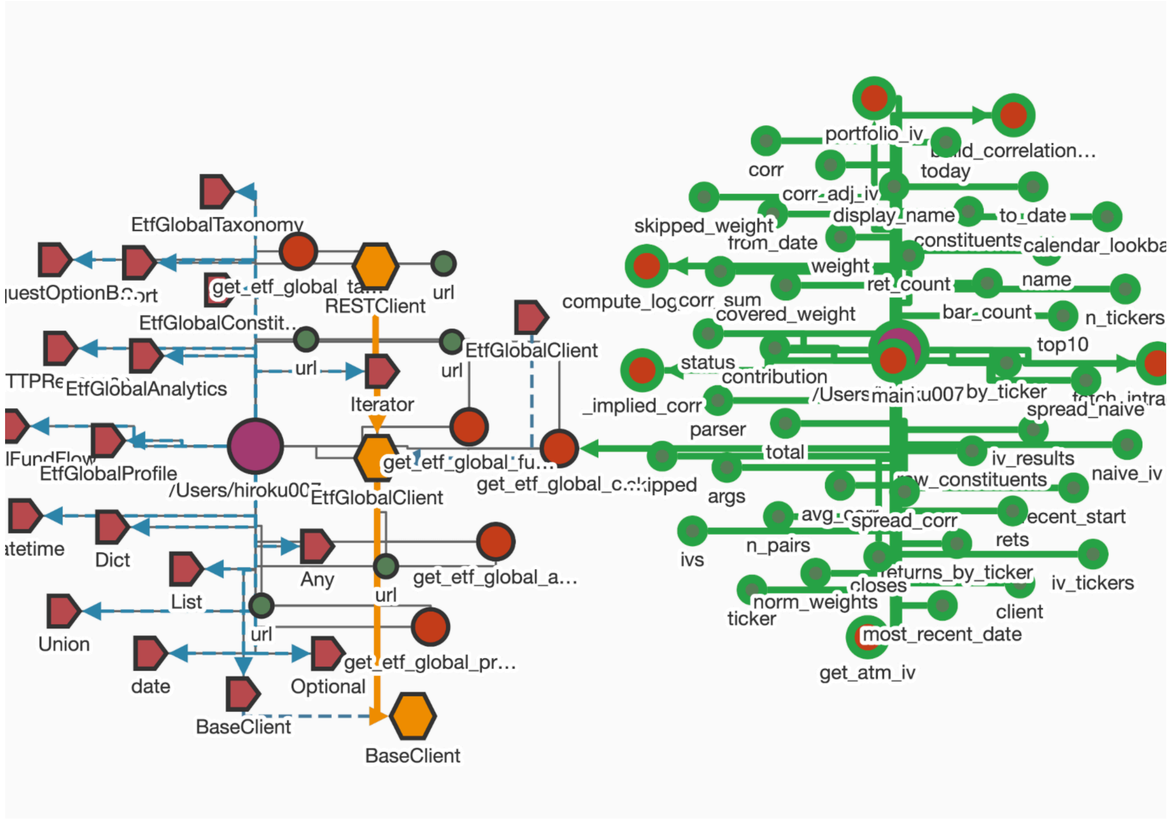
The subsample highlighted green are proposals

- Task:

Build a Python tool that compares QQQ's market-implied volatility to the theoretical IV derived from its weighted constituents.
The `client-python/` directory contains the Massive API Python client.
The API key is available as the MASSIVE_API_KEY environment variable.
Requirements:

1. Use the Massive client library (don't call the API directly — use the existing client classes)
2. Fetch QQQ's options chain and extract the at-the-money implied volatility for a user-specified expiry date
3. Fetch QQQ's constituent holdings and their weights
4. For each constituent with options available, fetch its ATM implied volatility for the same expiry
5. Calculate the weighted-average constituent IV (theoretical QQQ IV)
6. Display: QQQ market IV, theoretical IV, the spread, and a breakdown of the top 10 constituent contributions
7. Let the user specify the expiry date as a command-line argument
8. Handle gracefully: constituents without options data, API errors, rate limits

Create the tool as a standalone script (or small package) alongside the client-python directory. Include a README explaining how to run it.



Example use

Results

- Task:

Build a Python tool that compares QQQ's market-implied volatility to the theoretical IV derived from its weighted constituents.
The `client-python/` directory contains the Massive API Python client.
The API key is available as the MASSIVE_API_KEY environment variable.
Requirements:

1. Use the Massive client library (don't call the API directly — use the existing client classes)
2. Fetch QQQ's options chain and extract the at-the-money implied volatility for a user-specified expiry date
3. Fetch QQQ's constituent holdings and their weights
4. For each constituent with options available, fetch its ATM implied volatility for the same expiry
5. Calculate the weighted-average constituent IV (theoretical QQQ IV)
6. Display: QQQ market IV, theoretical IV, the spread, and a breakdown of the top 10 constituent contributions
7. Let the user specify the expiry date as a command-line argument
8. Handle gracefully: constituents without options data, API errors, rate limits

Create the tool as a standalone script (or small package) alongside the client-python directory. Include a README explaining how to run it.

Results without KG

```
==================================================================
  QQQ IMPLIED VOLATILITY COMPARISON
==================================================================
  Expiry:                2026-04-17
  QQQ Price:             $584.98
  Holdings Date:         2026-04-01
  QQQ Market IV:         26.09%
  Theoretical IV:        41.38%
  Spread (Mkt - Theo):   -15.29%
  Constituents covered:  101/101 (99.9% of weight)
------------------------------------------------------------------
```

```
/cost
└ Total cost:            $2.74
  Total duration (API):  6m 10s
  Total duration (wall): 40m 16s
  Total code changes:    364 lines added, 55 lines removed
  Usage by model:
      claude-haiku-4-5:  717 input, 9.4k output, 1.3m cache read, 70.4k cache write ($0.2613)
       claude-opus-4-6:  55 input, 17.7k output, 1.9m cache read, 173.0k cache write ($2.48)
```

Example use

Results

- Task:

Build a Python tool that compares QQQ's market-implied volatility to the theoretical IV derived from its weighted constituents.
The `client-python/` directory contains the Massive API Python client.
The API key is available as the MASSIVE_API_KEY environment variable.
Requirements:

1. Use the Massive client library (don't call the API directly — use the existing client classes)
2. Fetch QQQ's options chain and extract the at-the-money implied volatility for a user-specified expiry date
3. Fetch QQQ's constituent holdings and their weights
4. For each constituent with options available, fetch its ATM implied volatility for the same expiry
5. Calculate the weighted-average constituent IV (theoretical QQQ IV)
6. Display: QQQ market IV, theoretical IV, the spread, and a breakdown of the top 10 constituent contributions
7. Let the user specify the expiry date as a command-line argument
8. Handle gracefully: constituents without options data, API errors, rate limits

Create the tool as a standalone script (or small package) alongside the client-python directory. Include a README explaining how to run it.

Results with KG

Metric	Value
QQQ Market IV (ATM)	25.99%
Theoretical IV (weighted)	41.22%
Spread	-15.23%
Coverage	101/101 constituents (99.9% weight)

```
> /cost
  └ Total cost:            $1.99
    Total duration (API):  4m 27s
    Total duration (wall): 21m 34s
    Total code changes:    368 lines added, 49 lines removed
    Usage by model:
        claude-haiku-4-5:  624 input, 19 output, 0 cache read, 0 cache write ($0.0007)
         claude-opus-4-6:  53 input, 13.7k output, 2.0m cache read, 104.5k cache write ($1.99)
```

The heads up comparison

	With KG	Without KG
Cost	$1.99	$2.74
Time	21m36s (it made 1 mistake)	40m16s (it made 2 costly mistakes)
Total Code Changes	368 lines added, 49 lines removed	364 lines added, 55 lines removed

Conclusions

- Increase the speed with which a company's tech platform is improved
  - Improved agentic coding workflow leads to fewer errors, more easily understood iterations, and can drive rapid development
- Decrease the costs of maintaining a technology platform
  - Improved codebase representation improves relevant context awareness, reducing token and time costs
- Improve the function of the technology platform to meet changing business needs
  - Better ability to guide agents in coding to deliver directly towards your business's values